Progetto Digital Image Processing

Lorenzo Bogna
731281

Fasi del progetto

- Realizzione di un rilevatore di bordi basato su CNN.
- Produzione di una edge map probabilistica: ogni pixel ha una probabilità 0–1 di essere bordo.
- Uso di un post-processing Canny-like modificato per collegare i punti e colmare I gap.
- Riconoscimento di figure distine (linee chiuse formate da linked edges).

Dataset, training e versioni della rete

Dataset e target

- BSDS500: immagini naturali con annotazioni umane di bordo.
- Le annotazioni vengono mediate per ottenere target morbidi in [0,1].
- È stato aggiunto un piccolo set sintetico con figure geometriche.
- Input RGB ridimensionato a 320×320.

Training

- Loss function: weighted BCE + Dice Loss, con maggiore peso assegnato ai pixel di bordo per gestire lo sbilanciamento edge/background.
- Optimization: aggiornamento dei pesi tramite AdamW e backpropagation sulla loss calcolata pixel-wise tra predizione e ground truth.

TinyEdgeNet v1

- Architettura leggera encoder–decoder basata principalmente su depthwise separable convolutions.
- Tre livelli di downsampling con skip connection nel decoder per recuperare l'informazione spaziale.

TinyEdgeNet v2

- Architettura più capiente, con più canali, blocchi convoluzionali standard e bottleneck residuale.
- Decoder con skip connection e stadio finale di refinement ad alta risoluzione per migliorare la precisione dei bordi.

Alcune metriche

Modello	Precision	Recall	F1
TinyEdgeNet V1	0.6536	0.7593	0.7025
TinyEdgeNet V2	**0.6787**	**0.7403**	**0.7082**

Canny-like modificato: input e candidati

Input usati dal post-processing

- edge_prob: probabilità prodotta dalla CNN, non semplice intensità dell'immagine.
- g: magnitudine del gradiente calcolata sull'immagine originale.
- θ: orientazione del gradiente, cioè la normale locale al bordo.

Classificazione dei pixel

- strong edge: p ≥ high_prob.
- weak edge: low_prob ≤ p < high_prob.
- i pixel sotto low_prob vengono scartati.
- Iweak non sono accettati subito: devono rispettare connessione, g e θ.

Calcolo geometrico

$g = \sqrt{(gx^2 + gy^2)}$

$\theta = \text{atan2}(gy, gx)$

tangente del bordo = $\theta + \pi/2$

Valori di riferimento

- low_prob = 0.20, high_prob = 0.50
- min_gradient = 0.05
- max_angle_diff = 35°

Canny-like: orientation_aware_linking

Cosa fa il metodo

- Parte dai pixel strong (p ≥ high_prob) e li inserisce in uno stack.
- Propaga sul vicinato 8-connesso: un weak pixel è accettato solo se g ≥ min_gradient e θ è compatibile con quello del pixel già accettato.
- La differenza angolare è π-periodica: un bordo ha direzione, non verso.
- Il filtro su g è precalcolato sull'intera mappa weak: stesso risultato, un test in meno nel ciclo.
- Nuova opzione require_source_gradient: applica la soglia su g anche al pixel sorgente.

Il codice in breve

```
strong = prob >= high_prob
weak   = (prob >= low_prob) & ~strong

# un weak con gradiente debole non serve a nulla
weak &= (g >= min_gradient)

accepted = strong.copy()
stack    = lista dei pixel strong

while stack:
    y, x = stack.pop()

    for yy, xx in vicini_8(y, x):

        if accepted[yy, xx]:
            continue
        if not weak[yy, xx]:
            continue

        # orientazioni compatibili?
        if angle_diff(theta[y, x],
                      theta[yy, xx]) > max_diff:
            continue

        accepted[yy, xx] = True
        stack.append((yy, xx))

return accepted
```

Canny-like: bridge_small_gaps

Cosa fa il metodo

- Etichetta le componenti connesse della mappa di bordi.
- Esplora i due versi della tangente θ + π/2, distinti con dir_diff (2π-periodica).
- Scarta i candidati della propria componente
- Tiene il candidato più vicino nel cono di ±25°, con tangente compatibile entro 30°.
- Applica i ponti dal più corto, con union-find: uno solo per coppia di componenti.

Il codice in breve

```
n_lab, lab = cv2.connectedComponents(edges, 8)
ponti = []

for y, x in pixel_di_bordo:
    tangente = theta[y, x] + pi / 2
    for verso in (0, pi):
        direzione = tangente + verso
        migliore, d_min = None, inf
        for yy, xx in finestra(y, x, max_gap):
            # gia' collegati: salta
            if lab[yy, xx] == lab[y, x]:
                continue
            d = hypot(xx - x, yy - y)
            if d > max_gap or d >= d_min:
                continue
            phi = arctan2(yy - y, xx - x)
            # dentro il cono, nel verso giusto
            if dir_diff(direzione, phi) > cono:
                continue
            # tangente del candidato coerente
            if angle_diff(theta[yy, xx] + pi / 2,
                          phi) > max_diff:
                continue
            d_min, migliore = d, (yy, xx)
        if migliore:
            ponti.append((d_min, (y, x), migliore))

# dal piu' corto, uno per coppia di componenti
for d, (y, x), (yy, xx) in sorted(ponti):
    if unione(lab[y, x], lab[yy, xx]):
        cv2.line(out, (x, y), (xx, yy), 1, 1)  # (col, riga)
```

Canny-like: extract_contours

Cosa fa il metodo

- Trasforma la mappa linked_edges in una maschera binaria ed estrae i contorni.
- findContours traccia il perimetro dei tratti di bordo, non le regioni racchiuse: un tratto aperto ha area quasi nulla e cade sotto min_area.
- min_length filtra su arcLength e conserva i bordi aperti.
- Retrieval_mode è un parametro: RETR_CCOMP o RETR_TREE per i contorni annidati.
- color_contours disegna ogni contorno con un colore deterministico.

Il codice in breve

```
binary = (linked_edges > 0).astype(np.uint8) * 255

contours, _ = cv2.findContours(
    binary,
    retrieval_mode,
    cv2.CHAIN_APPROX_SIMPLE,
)

# toglie i frammenti piccoli
if min_area > 0:
    contours = [c for c in contours
                if cv2.contourArea(c) >= min_area]

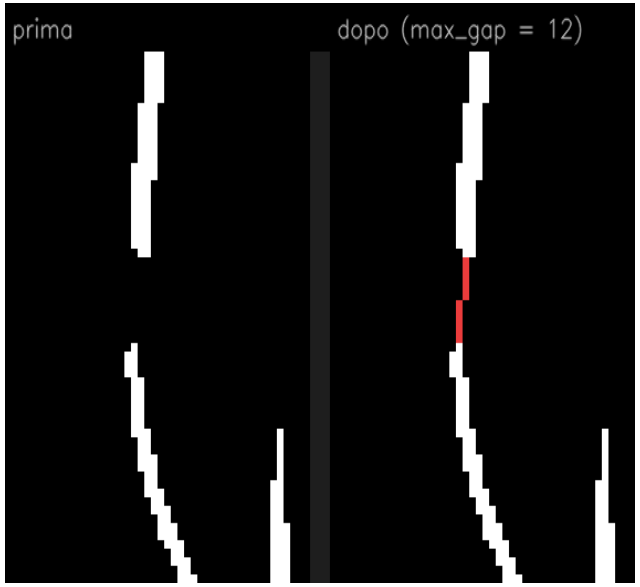
# alternativa: conserva i bordi aperti
if min_length > 0:
    contours = [c for c in contours
                if cv2.arcLength(c, False) >= min_length]

return contours
```

Bridging

Differenza con lunghrzza del bridge (3px vs 12px)

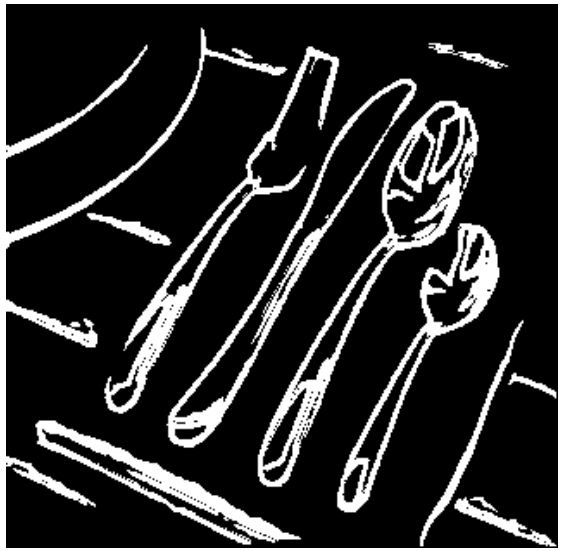




Bridging

Si nota come ne l coltello trmite il bridhe si chiude tutto il gap, così come nel cucchiao





Canny-like: link_cnn_edges

Cosa fa il metodo

- È la funzione che coordina l'intero post-processing Canny-like.
- Converte l'immagine in scala di grigi e calcola gx, gy, g e θ sull'originale.
- Combina probabilità CNN, gradiente e orientazione con orientation_aware_linking.
- Applica poi bridge_small_gaps con soglie angolari proprie
- Restituisce i campi: g, θ, gx, gy e linked_edges.

Il codice in breve

```
gray = cv2.cvtColor(image_bgr,
                    cv2.COLOR_BGR2GRAY)

# gradiente e orientazione dall'immagine vera
g, theta, gx, gy = gradients(gray, sigma)

linked = orientation_aware_linking(
    edge_prob, g, theta,
    low_prob, high_prob,
    min_gradient, max_angle_diff_deg,
)

if bridge_gaps:
    # soglie proprie, non quelle del linking
    linked = bridge_small_gaps(
        linked, theta,
        max_gap=max_gap,
        search_angle_deg=bridge_search_angle_deg,
        max_angle_diff_deg=bridge_max_angle_diff_deg,
    )

return {
    "g": g, "theta": theta,
    "gx": gx, "gy": gy,
    "linked_edges": linked,
}
```

Vantaggi rispetto a Canny classico

Canny classico

- Richiede soglie manuali e sensibili al rumore.
- Non usa conoscenza appresa dai dati.
- Può frammentare bordi deboli o collegare texture non utili.
- Restituisce linee, non figure: nessuno stadio successivo le ricompone.

Metodo proposto

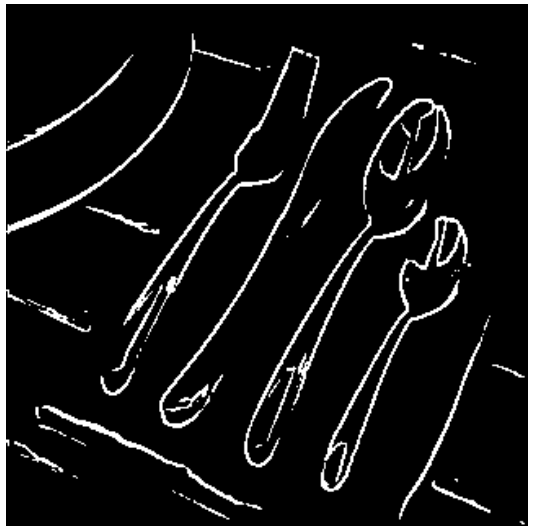
- La CNN apprende una probabilità di bordo da esempi reali e sintetici.
- Le soglie lavorano sulla probabilità, non solo sull'intensità.
- g conferma la presenza di una transizione locale reale.
- θ vincola la continuità geometrica dei segmenti.
- Il linking chiude i contorni: l'uscita si legge come figure separate.
- Il sistema di inferenza lavora molto rapidamente, compiendo le valutazioni con una media di 44 ms a immagine

Esempio: immagine naturale con oggetti

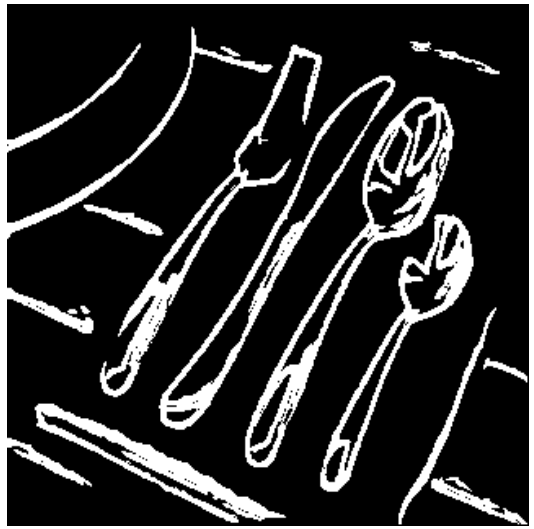
Probabilità CNN



Soglia



Linking



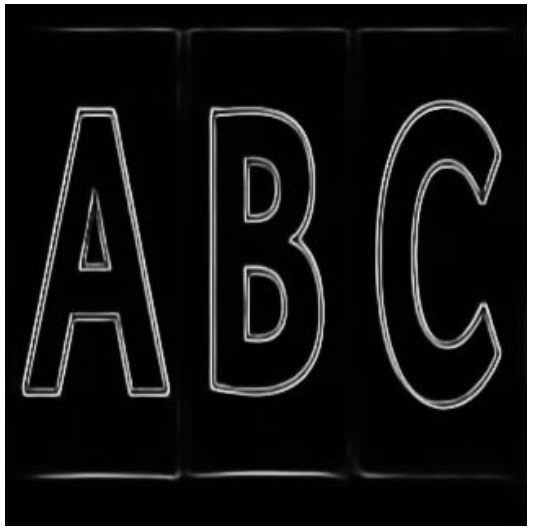
Contorni



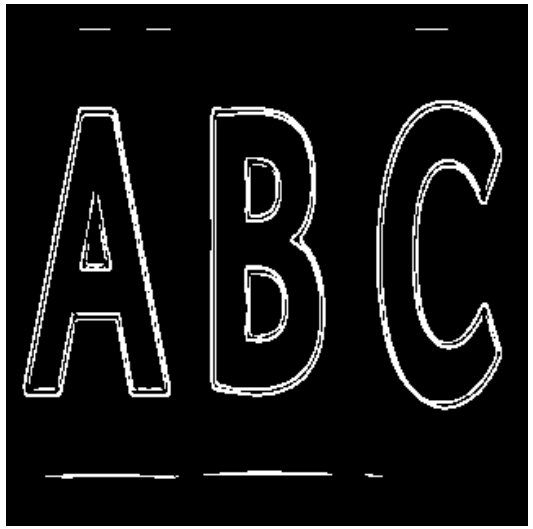
- La probabilità conserva informazioni continue sui bordi.
- La soglia seleziona i candidati, ma non basta a garantire continuità.
- Il linking orientato usa g e θ per unire segmenti coerenti.
- I contorni finali rendono più leggibile la separazione delle figure.

Esempio: lettere sintetiche

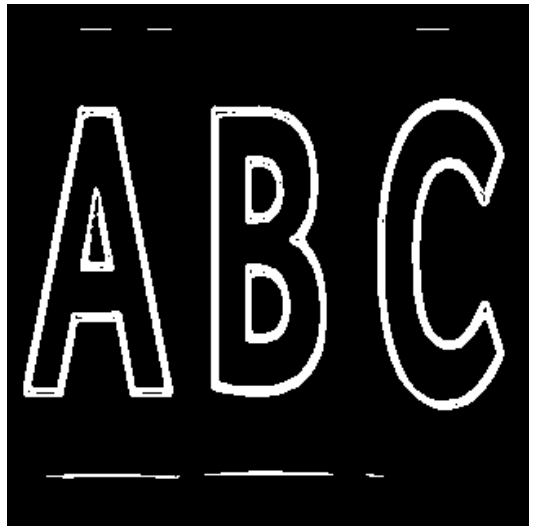
Probabilità CNN



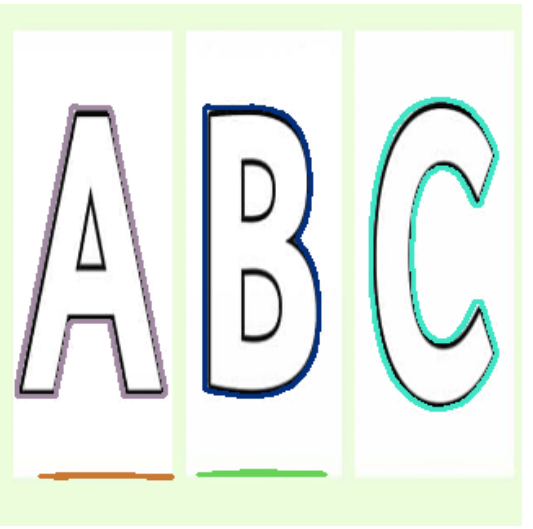
Soglia



Linking



Contorni



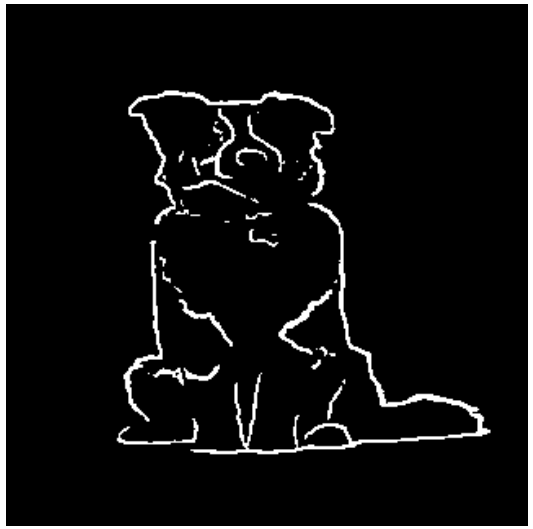
- La probabilità conserva informazioni continue sui bordi.
- La soglia seleziona i candidati, ma non basta a garantire continuità.
- Il linking orientato usa g e θ per unire segmenti coerenti.
- I contorni finali rendono più leggibile la separazione delle figure.

Esempio: cane

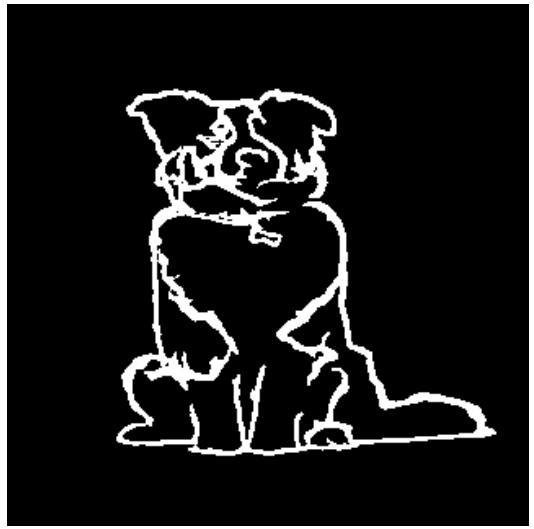
Probabilità CNN



Soglia



Linking



Contorni



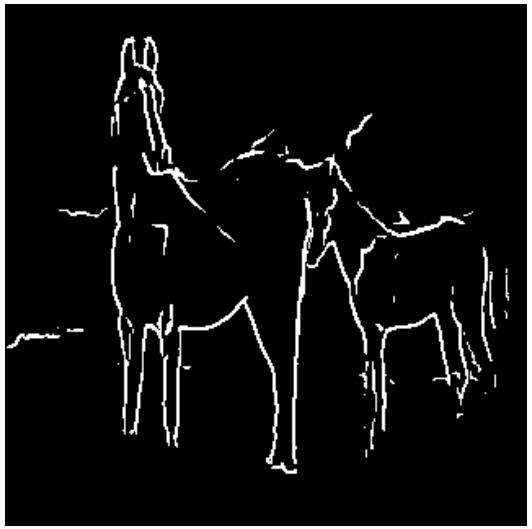
- La probabilità conserva informazioni continue sui bordi.
- La soglia seleziona i candidati, ma non basta a garantire continuità.
- Il linking orientato usa g e θ per unire segmenti coerenti.
- I contorni finali rendono più leggibile la separazione delle figure.

Esempio: cavalli

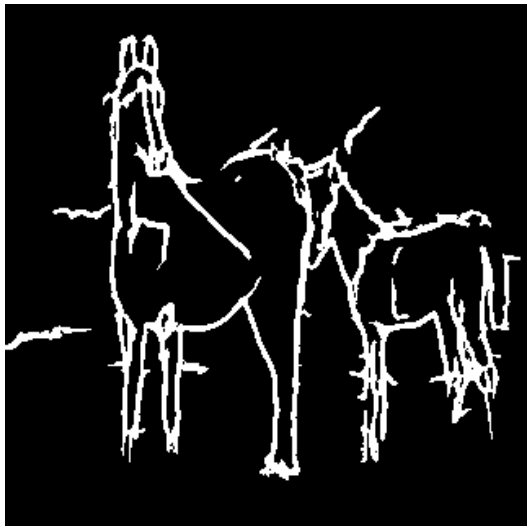
Probabilità CNN



Soglia



Linking



Contorni



- La probabilità raggiunge 0.92 sui profili degli animali e resta bassa sull'erba.
- La soglia accende il 4.2% dei pixel; dopo il linking si arriva al 9.8%.
- I bordi risultanti formano 9 componenti connesse, una delle quali raccoglie il 91% dei pixel.
- extract_contours con min_area = 10 restituisce 7 contorni.

Esempio: casa

Probabilità CNN



Soglia



Linking

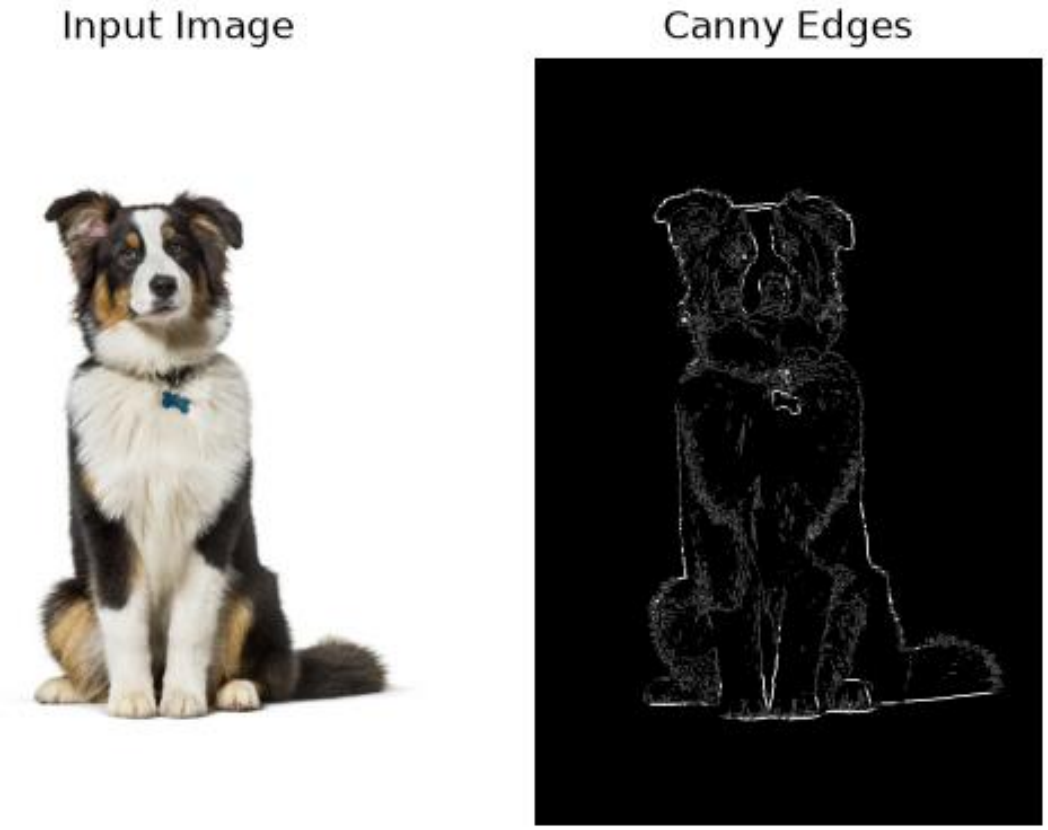


Contorni



Canny classico: soggetto su fondo uniforme

Risultato di cv2.Canny



Cosa mostra il confronto

La silhouette è ricostruita bene: contro uno sfondo uniforme il gradiente è netto e le soglie di isteresi bastano.

Dentro il pelo compare una trama fitta di micro-bordi: Canny risponde alla tessitura, non alla forma dell'oggetto.

I contorni del muso e delle zampe restano frammentati e non esiste un criterio per ricongiungerli.

L'uscita è binaria: non dice quanto un bordo sia plausibile, quindi non è filtrabile a valle.

TinyEdgeNet assegna a ogni pixel una probabilità appresa: la tessitura del pelo riceve valori bassi e cade sotto low_prob.

Il linking orientato ricongiunge poi solo i segmenti con g e θ coerenti, riconoscendo il contorno della figura.

Canny classico: scena con texture

Input Image



Canny Edges



I profili delle posate si interrompono dove il riflesso sul metallo annulla il contrasto locale.

Il bordo del coltello rimane aperto, a differenza dell'esempio di canny modificato che lo chiude

Alzare le soglie per togliere la tessitura cancella anche i bordi deboli delle posate: il gradiente da solo non distingue i due casi.

Nel metodo proposto la soglia agisce sulla probabilità CNN, mentre g resta come conferma che la transizione esista davvero.

Canny classico: scene naturali

Input Image



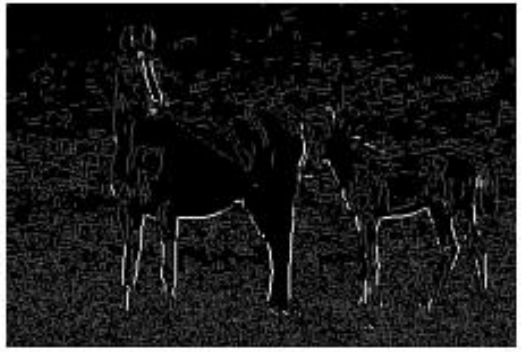
Canny Edges



Input Image



Canny Edges



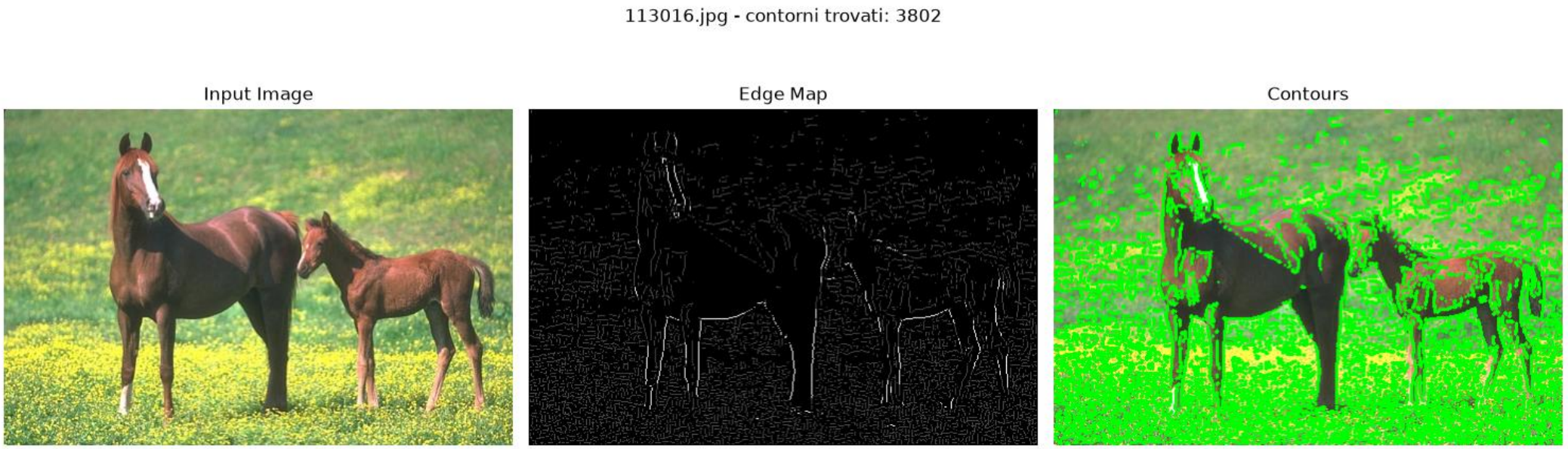
In entrambi gli output della canny normale c'è molto rumore causato dal prato.

La staccionata e il dorso dei cavalli non viene riconosciuto.

Canny basa la decisione su informazione locale di gradiente e hysteresis, senza conoscenza appresa dai dati.

La rete distingue invece un contorno da una tessitura, e il linking richiede che g e θ restino coerenti lungo il tratto.

Canny classico con find contours di cv2



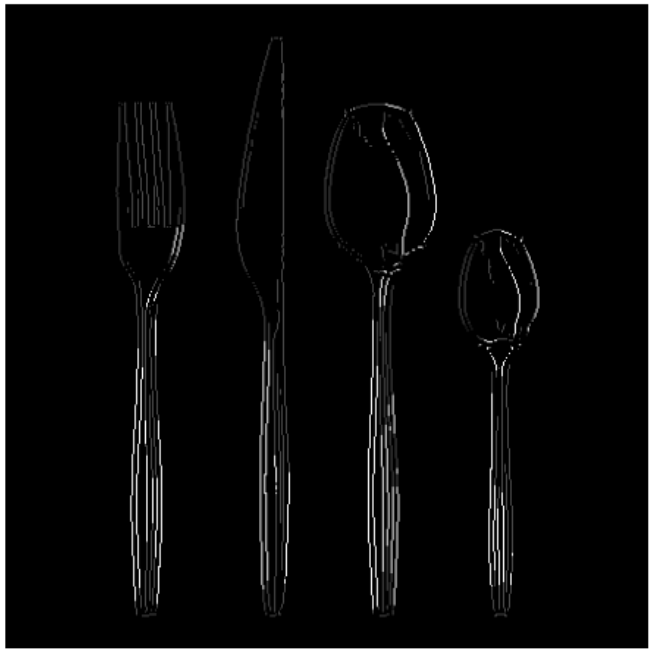
Come si può notare, a differenza del canny modificato, applicando contorni al canny classico vi sono problemi sia col rumore dello sfondo che con la sagoma delle figure

Canny classico con find contours di cv2

Input Image



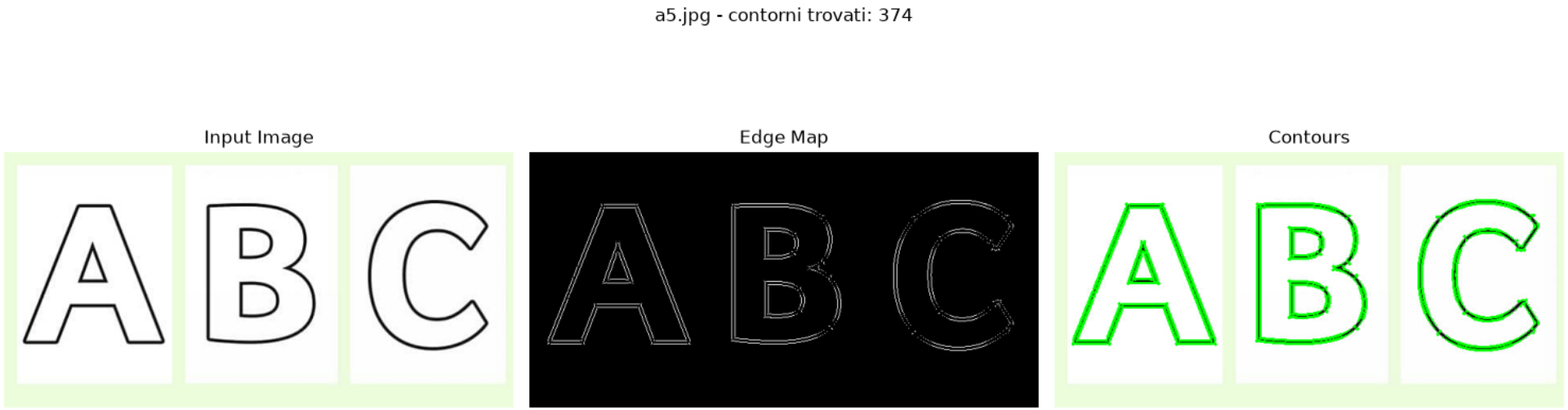
a10.jpg - contorni trovati: 230
Edge Map



Contours



Canny classico con find contours di cv2



Si nota facilmente, sia in questa diapositiva che in quella precedente, come figure senza sfondi producano risultati migliori, ma lasciano importanti buchi nel contorno che il canny modificato, grazie alle sue ottimizzazioni, non lascia

Inferenza: esecuzione su nuove immagini

Cosa fa il metodo

- Seleziona automaticamente il device disponibile: MPS, CUDA o CPU.
- Legge l’immagine, la ridimensiona e la converte in input RGB normalizzato per la rete.
- Carica i pesi, mette il modello in eval e produce la probability map con sigmoid.
- Applica link_cnn_edges, poi extract_contours e color_contours.
- Salva probabilità CNN, soglia, linked edges e contorni colorati.